Room - LLM Security

- Understand **LLM-specific vulnerabilities** that do *not apply* to traditional ML systems.
- Recognise how **LLMs create new attack surfaces** through natural language interfaces and memory/context mechanisms.
- Identify and categorise **LLM threat types**: data-based, model-based, system-based, user-based.
- Understand the **realistic risks** LLMs introduce in production systems.

Main Data-Based Threats

1. Training Data Extraction

Attackers try to recover actual text from the model's training data.

- They use specially crafted prompts to make the model reproduce memorized content.
- Extracted data may include:
  - Personal information (PII)
  - Emails
  - SSH keys
  - Confidential text
- Attackers identify memorized sequences by checking:
  - High confidence outputs
  - Repeated deterministic responses
  - Realistic structured data

Goal: Leak confidential training data.

---

2. Membership Inference

Attackers test whether a specific data sample was used during training.

- The attacker already possesses the sample.
- They query the model and observe:
  - Confidence scores
  - Prediction accuracy
  - Perplexity/loss
- Models usually respond more confidently to data they have seen before.

Goal: Determine whether certain private data was included in training.

---

3. Prompt Leakage (System Prompt Leakage)

LLMs use hidden system/developer prompts that contain instructions and business logic.

- Attackers try to trick the model into revealing these hidden prompts.
- Since the model treats system prompts and user prompts as part of one conversation history, prompt injection can expose them.
- Example:
    - Microsoft Bing AI ("Sydney") leaked its confidential system prompt in 2023.

Risks:

- Exposure of proprietary business logic
- Revealing safety mechanisms
- Helping attackers craft better prompt injection attacks

Mitigation:

- Never store secrets, API keys, or credentials inside system prompts.
- Do not treat system prompts as secure boundaries.

Key Difference Between Extraction and Membership Inference

Threat	Purpose	Output
Training Data Extraction	Recover actual hidden training data	Verbatim/near-verbatim content
Membership Inference	Check if known data was used in training	Yes/No or probability result

1. Model Extraction (Model Theft)

Model extraction is the unauthorized copying of an AI model.

- Attackers interact with the model through APIs.
- They send many prompts and store the input-output pairs.
- These pairs are then used to train a **surrogate model** that imitates the original model.
- In some cases, attackers may even recover the model's decision boundaries or weights.

Impact

- Major economic loss because high-quality LLMs require huge investments in:
    - Data
    - Training time
    - Computing resources
- Example:
    - Researchers from Mindgard recreated a smaller version of ChatGPT 3.5 Turbo using only about $50 in API queries.

In Short

- **Target:** Model parameters/intellectual property
- **Input:** Large numbers of API queries
- **Output:** A copied or distilled version of the original model

---

2. Model Inversion

Model inversion attacks try to reconstruct hidden training data from the model.

- Attackers analyse the model's outputs and internal representations.
- They iteratively optimize inputs to recover realistic training samples.
- Unlike membership inference, the attacker does **not** already know the data.

Difference from Membership Inference

Membership Inference	Model Inversion
Checks if known data was used in training	Reconstructs unknown training data
Output is yes/no	Output is recovered information
Tests model familiarity	Reverses learned information

Impact

- Privacy violations
- Leakage of confidential training data
- Exposure of sensitive attributes

Real Example

- In 2023, researchers extracted verbatim pieces of ChatGPT's training data using inversion-style techniques.

In Short

- **Target:** Internal model representations
- **Input:** Model outputs/embeddings
- **Output:** Reconstructed training data or hidden attributes

---

Key Idea

- **Model Extraction** steals the *model itself*.
- **Model Inversion** steals the *data hidden inside the model*.

1. Prompt Injection

Attackers manipulate the LLM's input context to override developer instructions.

- LLMs process all tokens equally inside the context window.
- The model cannot reliably distinguish:
    - Trusted system instructions
    - Untrusted user input
- Attackers craft malicious prompts to:
    - Bypass policies
    - Change model behaviour
    - Trigger unintended actions

In Short

- **Target:** Context window/instruction hierarchy
- **Input:** Malicious prompts or injected text
- **Output:** Altered behaviour or policy bypass

---

2. Context Overflow (Unbounded Consumption)

Attackers overload the model with huge amounts of text.

- LLM context windows have token limits.
- When the limit is exceeded:
    - Older tokens are removed (FIFO behaviour)
    - Important safeguards/system prompts may disappear
- This can cause:
    - Loss of protections
    - Denial of service
    - Increased API costs

Mitigation

- Rate limiting
- Token budgets
- Cost monitoring/alerts

Related Threat

- **Denial of Wallet (DoW):**
  Flooding APIs with oversized prompts to increase operational costs.

In Short

- **Target:** Context window and system resources
- **Input:** Extremely large prompts/documents
- **Output:** Truncated safeguards, degraded performance, higher costs

---

3. Memory Poisoning

Attackers corrupt long-term conversation memory.

- Stateful chatbots remember previous interactions.
- Attackers gradually insert false or malicious information.
- The poisoned memory affects future responses.

Example

If the chatbot is repeatedly told:

> “A cat is a dog”

it may later produce incorrect answers based on that false memory.

In Short

- **Target:** Persistent conversation memory
- **Input:** Malicious information stored over time
- **Output:** Persistent misinformation or corrupted responses

---

Key Idea

These attacks exploit how LLMs manage context and memory:

- **Prompt Injection** → Manipulates instructions
- **Context Overflow** → Pushes safeguards out of memory
- **Memory Poisoning** → Corrupts long-term conversation state

1. LLM-Powered Social Engineering

LLMs improve phishing and scam attacks by generating highly realistic messages.

- Traditional phishing signs like:
  - Bad grammar
  - Spelling mistakes
  - Poor wording

    are no longer reliable indicators.
- AI can generate professional spear-phishing emails that imitate:
  - Executives
  - Colleagues
  - Trusted organisations

Attackers can make these scams even stronger by combining:

- Stolen company/customer data
- OSINT (publicly available information)
- Information leaked from compromised LLMs

In Short

- **Target:** Human decision-making
- **Input:** Personal/contextual information
- **Output:** Manipulated users, phishing success, fraud

---

2. Trust Exploitation (Misinformation)

People often trust AI responses because they sound confident and authoritative.

- LLMs can hallucinate false information.
- Users may accept incorrect answers without verification.
- Attackers exploit this over-trust.

Example: Package Hallucination

- An AI coding assistant invents a fake package name like:
  `secure-utils-xtools`
- Attackers publish malware under that fake package name.
- Developers who trust the AI may unknowingly install malicious software.

This turns hallucinations from a reliability issue into a real security threat.

In Short

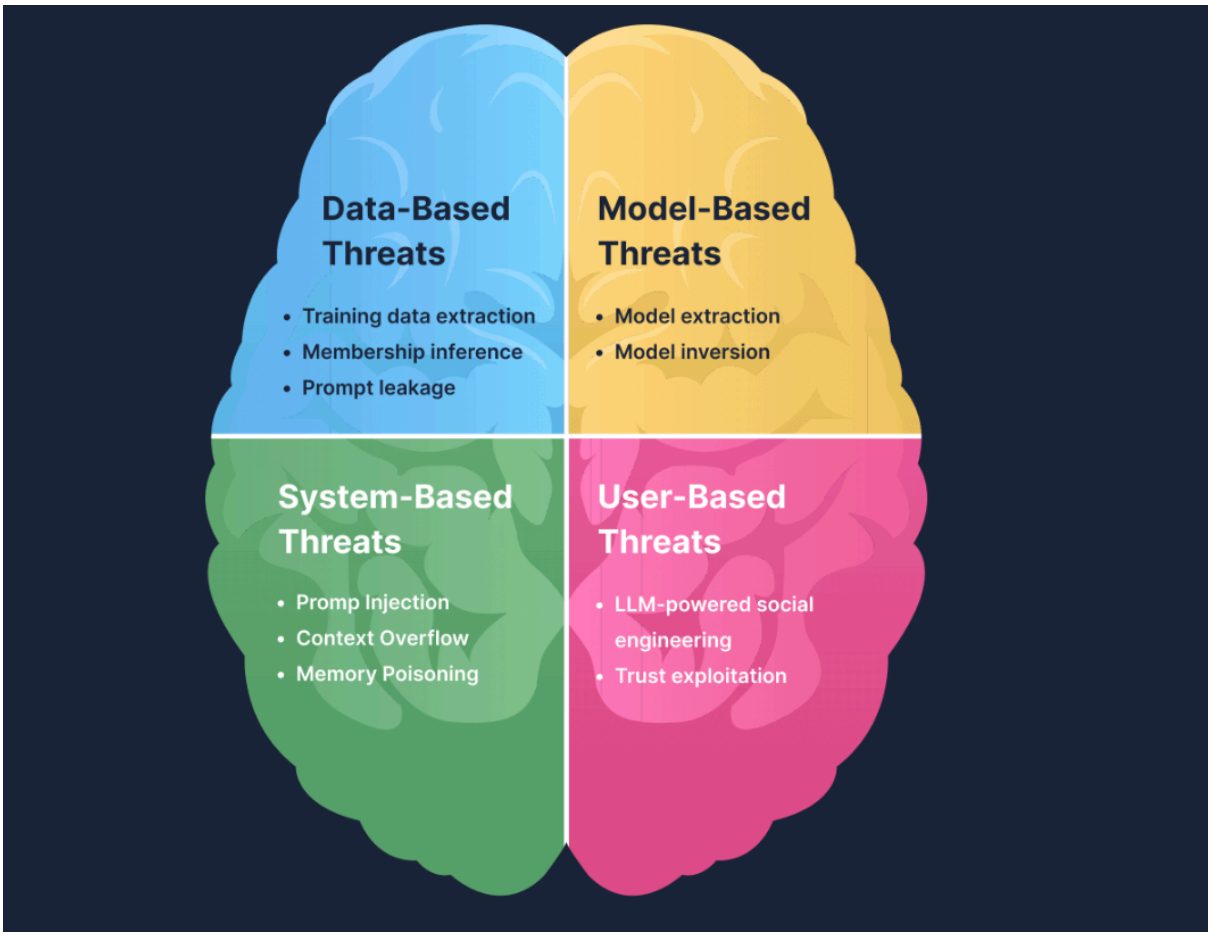
- **Target:** User trust and judgment
- **Input:** Confident but false or malicious AI output
- **Output:** Users accepting unsafe or harmful information

Key Idea

LLMs can be weaponized to manipulate humans by:

- Creating highly convincing phishing attacks
- Exploiting trust in AI-generated responses
- Spreading misinformation or malicious recommendations

This highlights the importance of always verifying AI-generated information instead of blindly trusting it.



Type	Threat	Target / Attack Surface	Input	Output
Data-Based	**Training Data Extraction**	Training dataset (confidentiality)	Crafted prompts designed to trigger memorised content	Verbatim or near-verbatim training data (text, PII, secrets)
Data-Based	**Membership Inference**	Training dataset membership (privacy metadata)	Known candidate data sample already possessed by the attacker	Yes/no (or probability) decision indicating whether the sample was used in training
Data-Based	**Prompt Leakage / System Prompt Exposure (LLM07:2025)**	System prompt / developer instructions	Prompts asking the model to reveal or reflect on its instructions	Partial or full disclosure of hidden system or developer prompts
Model-Based	**Weight Extraction (Model Stealing)**	Model parameters (intellectual property)	Large volumes of carefully chosen API queries	A surrogate or distilled model replicating the original model's behaviour
Model-Based	**Model Inversion**	Model's internal representations	Unknown or partially known data, or model embeddings/outputs	New training data or attributes reconstructed from the model
System-Based	**Context Window Poisoning (Prompt Injection)**	LLM context window (instruction hierarchy)	Attacker-controlled text embedded in input or retrieved content	Altered behaviour, policy bypass, unintended actions
System-Based	**Context Overflow / Unbounded Consumption (LLM10:2025)**	Context window size and system resources	Excessively large prompts or documents	Truncated safeguards, degraded responses, or denial of service
System-Based	**Stateful Conversation Manipulation (Memory Poisoning)**	Persistent conversation memory	Malicious statements intended to be stored as long-term context	Persistent misinformation or corrupted future responses

System-Based	**Context Overflow / Unbounded Consumption (LLM10:2025)**	Context window size and system resources	Excessively large prompts or documents	Truncated safeguards, degraded responses, or denial of service
System-Based	**Stateful Conversation Manipulation (Memory Poisoning)**	Persistent conversation memory	Malicious statements intended to be stored as long-term context	Persistent misinformation or corrupted future responses

| User-Based | **LLM-Powered Social Engineering** | Human cognition and decision-making | Contextual or personal information used to craft persuasive output | Manipulated users (phishing success, fraud, coerced actions) |
| User-Based | **Trust Exploitation / Misinformation (LLM09:2025)** | User trust and judgment | Confident but incorrect or maliciously framed prompts | Users accepting false, unsafe, or harmful information |

Key Takeaways

- **LLMs introduce a unique attack surface** distinct from traditional ML systems, driven by natural language interaction, context handling, and emergent behaviour.
- **Data-based threats** exploit how LLMs learn from and memorise training data, enabling attacks such as training data extraction, membership inference, and system prompt leakage.
- **Model-based threats** target the model itself, including model extraction (theft of model behaviour or weights) and model inversion (reconstructing sensitive training data).
- **System-based threats** arise from how LLMs process all inputs as a single context, enabling prompt injection, context window overflow, and memory poisoning.
- **User-based threats** leverage LLMs as force multipliers for social engineering, increasing the effectiveness of phishing, scams, and trust exploitation.